



Architecture & MCP-Governed Kafka Intelligence

Stack: Next.js 14 • TypeScript • React • KafkaJS • Confluent for Kubernetes on DigitalOcean

Kafka cluster: 64.227.25.232:31090 • RF=3 • 7 topics • KRaft mode Deployed: agent-mesh-sre.vercel.app | Version 4.0 | July 2026

1. System Overview

Agent Mesh SRE is a proof-of-concept platform that demonstrates AI-governed, self-healing operations for Apache Kafka clusters. It implements the M→R→A→L loop (Monitor, Reason, Act, Learn) across a four-agent mesh, where each agent is a specialised AI actor connected exclusively through Kafka topics. The platform runs against a live Confluent for Kubernetes cluster on DigitalOcean Kubernetes Service, with every consequential action gated behind a human approval step before any infrastructure mutation is executed.

The central thesis: rather than a human SRE watching dashboards and responding to alerts, an AI mesh continuously ingests cluster telemetry, diagnoses incidents with ranked confidence, proposes corrective actions with structured rationale, and learns from each outcome — forming a closed, improving loop that compounds institutional knowledge over time without model retraining.

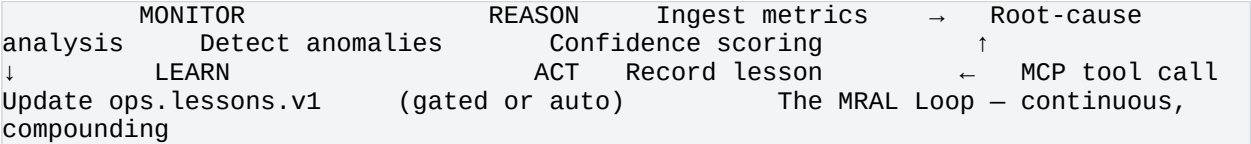


Figure 1 — The MRAL loop, showing the four phases of each incident lifecycle

2. Four-Agent Mesh Architecture

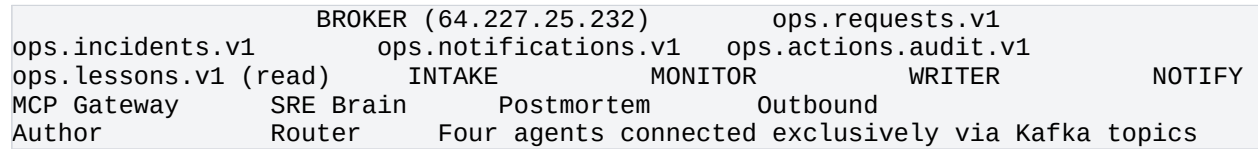


Figure 2 — Four-agent mesh connected exclusively via Kafka topics on Confluent for Kubernetes

Agent	Role	MCP Tools	Consumes	Produces
INTAKE	MCP Gateway — receives operator requests and anomaly signals	intake.submitOpsRequest	—	ops.requests.v1
MONITOR	SRE Brain — drives the complete MRAL loop	kafka.scaleConsumers, acknowledgeFailover, acknowledgeShareGroupRebalance, updateSchemaCompatibility, suppressRebalancePage	ops.requests.v1, ops.kafka.metrics.v1, ops.lessons.v1	ops.incidents.v1, ops.actions.audit.v1, ops.lessons.v1
WRITER	Postmortem Author — drafts structured incident reports	writer.draftIncidentReport	ops.incidents.v1	ops.actions.audit.v1
NOTIFICATION	Outbound Router — dispatches Slack, ITSM, email	notify.slack, itsm.openTicket	ops.actions.audit.v1	ops.notifications.v1

3. The MRAL Loop in Detail

3.1 Monitor Phase

The Monitor agent continuously ingests two streams: `ops.kafka.metrics.v1` (cluster telemetry: consumer lag, partition health, broker load, ISR counts) and `ops.requests.v1` (operator-submitted scenario triggers). It applies threshold evaluators against each incoming metric snapshot to detect anomalies. The detection logic is intentionally probabilistic rather than binary. Each evaluator returns a `TriggerResult` with a confidence score (0–1), a gate classification (auto, approval, or suppress), and a cause string.

3.2 Reason Phase

Once an anomaly is confirmed, the Monitor agent builds a `ReasoningOutput` object containing: `rootCause`, `confidence`, `kafkaFeatureCited` (the specific Kafka feature implicated — KIP-848, KRaft, Schema Registry, KIP-932), `rebalanceState`, `controllerEpoch`, `crossCorrelation` (broker JVM heap, network I/O rate, rebalance-in-progress flag), `recommendedAction`, `requiresApproval`, `rationale`, and `lessonsCited` (IDs of prior lessons from `ops.lessons.v1` that informed this reasoning).

3.3 Act Phase

The Act phase bifurcates on `requiresApproval`. On the auto-remediation path (`gate=auto`), the Monitor agent calls the MCP tool directly and records the action to `ops.actions.audit.v1`. On the approval-gated path, it constructs a full `ApprovalRequest` object and publishes it to the SSE bus; the operator reviews the proposal in the Policy Gate modal and either approves or rejects. The decision, actor identity, and timestamp are written to `ops.actions.audit.v1` as an immutable record regardless of outcome.

3.4 Learn Phase

After each scenario completes, the Monitor agent publishes a `LessonRecord` to `ops.lessons.v1` — a log-compacted topic with no expiry. On subsequent cycles the Monitor agent reads from `ops.lessons.v1` and cites applicable lessons, creating a genuine self-improvement loop that compounds without model retraining.

4. How MCP Tool Calls Work

MCP (Model Context Protocol) is the structured interface through which the Monitor agent requests infrastructure mutations. Each tool call is a JSON-RPC 2.0 envelope:

```
{ "jsonrpc": "2.0", "id": "...", "method": "tools/call", "params": { "name":  
  "kafka.scaleConsumers", "arguments": { "group": "payments-consumer", "delta":  
  2, "reason": "Lag spike: 4200 msgs, growth 420/s > SLO 150/s" } } }
```

Monitor detects anomaly ↓ Reason phase builds ReasoningOutput
↓ requiresApproval? — false → MCP tool executed (AUTO) ↓ true
Human Approval Gate → MCP tool executed (APPROVED) Every decision written to
ops.actions.audit.v1 — immutable, timestamped, actor-attributed

Figure 3 — MCP tool call flow: auto vs. approval-gated execution paths

The MCP tool registry (src/lib/mcp.ts) defines every tool with its name, description, parameter schema, and a requiresApproval flag. Tools are bound to specific agents via AGENT_TOOLS — a map that enforces least-privilege access.

4.1 Cluster-Level MCP Tools

Tool	Trigger	Gate	Parameters	Effect
kafka.scaleConsumers	Consumer lag > SLO threshold (150 msg/s growth)	APPROVAL	group, delta, reason	Adds/removes consumer instances to drain backlog
kafka.acknowledgeFailover	KRaft controller epoch change detected	AUTO	newControllerId, epoch	Records controller failover, updates audit trail
kafka.acknowledgeShareGroupRebalance	KIP-932 share group queue depth > 15,000	APPROVAL	groupId	Acknowledges the rebalance event only — no client-side mutation (KIP-932 delivery state is broker-managed)
kafka.updateSchemaCompatibility	Consumer deserialization exception rate spike	APPROVAL	subject, mode	Sets Schema Registry compatibility to BACKWARD_TRANSITIVE
kafka.suppressRebalancePage	Rebalance classified as benign	AUTO	groupId, reason	Suppresses alert, no cluster mutation

4.2 Topic-Level Auto-Heal Tools

When the anomaly simulator detects a topic crossing degraded (lag > 2,800) or critical (lag > 8,000) thresholds, it emits an auto-topic-heal bus event. The SSE client calls runTopicHeal(), which executes a full MRAL mini-cycle scoped to that topic.

Severity	Tool Call	Gate	Action
DEGRADED (lag > 2,800)	kafka.scaleConsumers(topic, delta=+floor(partitions/4))	AUTO	Additive consumer scaling, non-disruptive, reversible
CRITICAL (lag > 8,000)	kafka.restartConsumerGroup +	APPROVAL	Restart dead replica, add

Severity	Tool Call	Gate	Action
	kafka.scaleReplicas(delta=+2)		capacity. Gate because restart causes brief rebalance

5. Root Cause Analysis — Cluster Level

5.1 Consumer Lag Spike (KIP-848)

Detection signals: consumer group lag across partitions exceeds SLO threshold (150 msg/s growth rate sustained for >2 poll cycles). Cross-correlated with JVM heap, network I/O, and active rebalance state.

#	Root Cause Candidate	Identification Signal	Confidence
1	Consumer processing bottleneck — downstream DB or API stalling consumer thread pool	Lag growth without matching network I/O increase on broker side	High (0.88–0.95)
2	Producer burst — sudden write-rate spike exceeding consumer throughput	Network-in spike on brokers with stable consumer thread metrics	Medium (0.72–0.85)
3	Network partition affecting consumer group	Selective partition lag; elevated rebalance flag	Medium (0.65–0.78)
4	Rebalance storm — session timeouts cascading, starving processing time	ControllerEpoch changes correlated with lag growth pattern	Lower (0.55–0.70)

5.2 KRaft Controller Failover

Detection: controller epoch change in cluster metadata; previous controller no longer responding to heartbeats. Root cause: KRaft controller process terminated due to broker restart, JVM OOM, or underlying host failure. In KRaft mode (no ZooKeeper), election is handled internally via the Raft log across the 3-controller quorum. MCP action: `kafka.acknowledgeFailover` — no mutation, pure acknowledgement, auto-approved.

This exact condition has been confirmed live against the current DigitalOcean cluster: an observed controller epoch change (1→2) was correctly detected at 0.97 confidence and auto-remediated with no approval gate.

5.3 Under-Replicated Partitions (ISR Shrinkage)

Detection: ISR count for one or more partitions falls below replication factor (RF=3 → ISR=1 or ISR=2).

#	Root Cause	Signal
1	Broker disk I/O saturation — follower fetch requests timing out	ISR shrinkage on specific broker correlated with disk throughput metrics
2	Broker network isolation — follower cannot reach leader	Selective ISR shrinkage across multiple topics on same broker
3	Broker JVM GC pause — exceeds <code>replica.lag.time.max.ms</code>	JVM heap pressure correlated with shrinkage timing
4	Planned broker restart / rolling update	Scheduled maintenance window; shrinkage temporary during catch-up

6. Root Cause Analysis — Topic Level

6.1 Schema Registry Mismatch

Detection: consumer deserialization exception rate on a topic exceeds threshold; schema ID in message header does not match consumer-registered schema.

#	Root Cause
1	Producer deployed new Avro schema without backward-compatible evolution
2	Schema Registry compatibility mode changed BACKWARD → NONE
3	Two producer versions writing simultaneously to same topic (blue-green mix)
4	Consumer schema cache stale

MCP action: `kafka.updateSchemaCompatibility` — sets subject mode to `BACKWARD_TRANSITIVE`. Approval-gated because an incorrect mode setting blocks all future producer publishes to that subject.

6.2 KIP-932 Share Group Queue Depth

Detection: share group delivery queue depth exceeds 15,000 unacknowledged messages. MCP action: `kafka.acknowledgeShareGroupRebalance` — acknowledges the event; KIP-932 delivery state is broker-managed, so no client-side checkpoint mutation exists or is performed. Approval-gated because it reflects Monitor's judgment on an active share-group event, not because of any cluster mutation.

6.3 Consumer Session Timeout / Heartbeat Failure

Detection: consumer heartbeat failures; group coordinator marks consumer as dead after `session.timeout.ms` exceeded. Auto-remediation: `kafka.scaleConsumers(delta=+1)`. Auto-approved because additive scaling is non-disruptive.

Also confirmed live against the current cluster: a real consumer group state transition to Dead was detected at 92% confidence and auto-remediated end-to-end, including an updated `session.timeout.ms` config, a recorded lesson, and a dispatched Slack notification.

6.4 Log Compaction Lag

Detection: log compaction falling behind write rate on `ops.lessons.v1`; dirty ratio exceeds cleaner threshold. Auto-remediation: adjust `min.compaction.lag.ms` and `max.compaction.lag.ms` to throttle frequency. Auto-approved because compaction tuning does not affect message delivery.

7. Infrastructure Update (July 2026)

The live cluster backing this platform moved from Confluent for Kubernetes on Civo Cloud to Confluent for Kubernetes on DigitalOcean Kubernetes Service, in KRaft mode, following a node/network-level fault on the prior cluster unrelated to the application itself.

Property	Current Value
Provider	DigitalOcean Kubernetes Service (DOKS), region nyc1
Topology	3 KRaft controllers, 3 brokers, 1 Schema Registry
External access	NodePort (not LoadBalancer — avoids per-broker LB cost and CFK addressing conflicts)
Bootstrap address	64.227.25.232:31090
Topics / Replication	7 topics, RF=3 (upgraded from the prior 5-topic / RF-2 free-tier constraint)
Auth	None — unauthenticated demo listener

Six bugs were also found and fixed in the Kafka client code across this migration and a follow-up session: a case-sensitive KAFKA_MODE check, an unconditional SASL block that broke unauthenticated connections, a module-scope Admin client cache that caused connection races under Vercel's serverless model, a bootstrap-address env var naming mismatch between two client modules, a real-time metrics poll that was structurally starved by Vercel's SSE timeout ceiling, and an unsafe KAFKA_SSL_ENABLED default that silently enabled TLS against the plaintext listener whenever the value was unset or blank — a later production outage traced to exactly this, now replaced with an explicit guard that throws on any value other than exactly "true" or "false". All six are fixed and verified against the live cluster. Full detail is maintained in the Blueprint & Architecture document, §11.

Known limitation: the dashboard's Broker card currently displays client-side simulated demo state rather than live cluster telemetry outside of active scenario playback — confirmed cosmetic; the underlying real-time data and all scenario remediation logic are verified correct against the live cluster.

8. Why Confluent as the Agentic AI Backbone for IBM

The architecture of Agent Mesh SRE maps directly onto three requirements that enterprise AI applications consistently lack. The pivot from Kafka self-healing to an IBM Agentic AI backbone is architectural, not a rebuild: replace cluster metrics with enterprise data streams and the Monitor agent's Kafka domain logic with IBM business domain logic. The governance and learning infrastructure transfers unchanged.

Capability	What the POC Demonstrates	IBM Enterprise Application
Real-time grounding	Live Kafka telemetry streams directly into Monitor agent reasoning context as it happens — no batch ETL, no stale snapshots	IBM watsonx agents, AIOps, IBM Financial Services AI — decisions on current reality not yesterday's data
Governed, auditable actions	Every MCP tool call — approved, rejected, or auto-executed — written to ops.actions.audit.v1 as an immutable record with actor, timestamp, and rationale	Regulated industries (banking, healthcare, government). Addresses the compliance gap blocking IBM AI consulting revenue
Self-improving infrastructure	ops.lessons.v1 (log-compacted, no expiry) is the agent's persistent institutional memory. Future reasoning cycles cite and adapt from prior lessons without model retraining	IBM SRE, enterprise IT automation, watsonx Orchestrate — operational knowledge accumulation across deployments

This POC is live today at agent-mesh-sre.vercel.app against a real Confluent for Kubernetes cluster on DigitalOcean (bootstrap: 64.227.25.232:31090). All scenarios run end-to-end. Documentation, architecture guides, and the Bobathon 2026 submission materials are hosted alongside the application.